IUBAT— INTERNATIONAL UNIVERSITY OF BUSINESS AGRICULTURE AND TECHNOLOGY

Report

Course Code: CSE 3391

Course Name: Web Engineering

Submitted to

Name: Naeem Mia

Lecturer

Department of CSE.

Submitted by

ID	NAME
23103374	Md Mahfuzul Haque
23103380	Ehetisum Sharif
23103388	Nishat Tasnim
23103399	Md Sajid

Date of submission: 23 September 2025

Contents

Web Engineering: E-commerce System

1. Introduction

This part introduces the topic and sets the foundation for the report. It explains what web engineering is and why it's important, especially in the world of e-commerce.

1.1 Overview of Web Engineering

Web Engineering is the process of building and maintaining web-based applications in a proper, systematic way. It uses ideas from software engineering, information systems, and how people interact with computers. Web engineering ensures that applications are methodical, well-structured, and satisfy the objectives of both businesses and customers by the engineering concepts. The goal is to make web systems that are reliable, efficient, and easy to use.

1.2 Role of Web Engineering in Modern Applications

Web Engineering is crucial to modern technology as it drives digital transformation in almost every sectors. Web engineering helps make sure that websites are high-quality, secure, scalable, and easy to maintain. In business, it powers e-commerce sites, customer service systems, and stock management. In education, it facilitates online classrooms and digital learning platforms. In healthcare, it enables virtual consultations, electronic records, and appointment scheduling systems. In the domain of entertainment, it drives streaming services, gaming, and social media applications. These examples demonstrate how Web Engineering improves daily life by enabling essential services.

1.3 Importance of E-commerce in Web Environment

Global business operations have changed as a result of online commerce. It makes it possible for anyone to shop online whenever they want and from any place. This gives companies access to a larger market and makes it possible for them to offer specialized services. E-commerce is rapidly growing and now accounts for a sizeable portion of the global economy.

1.4 Objectives of the Report

This report's main goal is to analyze the foundational ideas of web engineering and show how to use them to create a safe, scalable, and reliable e-commerce platform. The paper will outline how to put these ideas into practice in order to create a system that meets user needs and operates efficiently.

1.5 Scope of the E-commerce System

This study will focus on the development of an e-commerce system with specific features. This includes setting up an account and signing in, browsing items, adding products to a cart,

making secure payments, and monitoring orders through a dashboard. The system will facilitate these main functions

1.6 Methodology

To prepare this report, a literature review was done. To understand the topic, this entails reading academic papers, industrial research, and technical literature. Mention the sources you used, including technical documentation, industry reports, and scholarly articles. Additionally, the report assesses existing e-commerce systems to determine their benefits and drawbacks.

2. Problem Statement

2.1 Limitations of Traditional Retail & Business Models

Restricted geographic reach: only local clients can be served via physical stores.
high operating cost: High operating expenses include inventory control, rent, employee compensation, and electricity.
Limited working hours: we are unable to offer round-the-clock assistance. Lack of personalization makes it challenging to monitor consumer preferences and make product recommendations. Manual procedures like stock management, billing, and record-keeping take a lot of time and are prone to mistakes.

2.2 Challenges in Web-based Shopping Platforms

- **Performance issues** – slow website speed, crashes during high traffic, poor customer experience.
- **Scalability –** difficulty in handling large numbers of users, products, and transactions as the business grows**.**
- **Security and privacy concerns** – risk of hacking, data breaches, identity theft, and misuse of personal data.
- **Payment reliability** – failed transactions, insecure payment gateways, limited access to trusted payment options**.**

2.3 Objectives of the Proposed Web-based E-commerce Solution

1. Create a dependable, quick, and easy-to-use internet purchasing platform.

2. Make sure it is scalable to accommodate future increases in transactions and traffic.

3. Put robust security mechanisms in place (HTTPS, SSL, encryption, and authentication).

4. Include a number of trustworthy payment methods (cards, online banking, and mobile wallets)

2.4 Success Criteria

Performance – fast page loading, reduced downtime, and consistent performance during high traffic.

scalability– capability to grow with additional users and products while maintaining quality.

Safety – adherence to industry benchmarks to safeguard confidential client information.

Client contentment – straightforward navigation, trustworthy payments, secure transactions, and favorable reviews

3. Background Research

3.1 History & Evolution of Web Engineering

Web Engineering has progressed considerably since the internet's inception. At first, web systems comprised straightforward static sites that offered merely essential information, frequently lacking interactivity or user involvement. Users were able to read content but unable to engage or contribute. The emergence of Web 2.0 transformed websites into dynamic and interactive platforms, enabling user engagement through social networking, comments, and sharing of content. This time marked an important change, highlighting user involvement, teamwork, and more engaging online experiences.

In recent years, contemporary web systems have progressed to encompass cloud services, mobile apps, AI integration, and advanced e-commerce platforms. These systems are extremely interactive, scalable, and able to manage significant volumes of data and numerous users at the same time. Consequently, Web Engineering has emerged as a field to oversee the design, development, and upkeep of these intricate web applications, guaranteeing they stay dependable, safe, and efficient

3.2 Principles of Web Engineering

Usability: All users should find the design to be straightforward, intuitive, and simple to use.
• **Interoperability**: online systems ought to be compatible with other platforms, devices, and applications.
• **security:** guard user information and system integrity against abuse or cyberattacks.

Scalability – systems should grow smoothly as users, products, or transactions increase**.**

3.3 Models of Web Development

Waterfall Model – This model utilizes a linear and sequential method, where each phase needs to be finished prior to proceeding to the subsequent one. It has reduced adaptability to changes, rendering it better suited for small or straightforward web projects with well-defined needs.

Agile Model – Agile is a flexible and repetitive methodology. It enables developers to operate in brief iterations and consistently incorporate user feedback to enhance the system. This makes it perfect for contemporary web development, where needs may often shift and quick delivery is essential.

Spiral Model – The Spiral Model focuses on managing risks. It merges prototyping with iterative development, enabling developers to assess and enhance the system during every phase. This framework is especially beneficial for extensive and intricate web initiatives that necessitate thorough planning, evaluation, and risk analysis

3.4 Analysis of Current E-commerce Websites

Amazon – worldwide frontrunner, great scalability, AI-based customization, effective logistics.

Flipkart – well-known in India, emphasizes discounts, localized services, and quick delivery.

Daraz – operates in South Asia, merging logistics with e-commerce for local markets.

Shopify – a platform for small enterprises, facilitates effortless setup of online shops.

Comparison of features – variety of products, design of UI/UX, options for payment, systems for delivery, and mechanisms for building customer trust.

3.5 Review of Literature on Web Technologies

Front-end Tools – HTML, CSS, JavaScript, and frameworks such as React are employed to build the segment of a web application that users experience and engage with. HTML structures web pages, CSS handles styling and layout, while JavaScript introduces interactivity. Frameworks such as React assist developers in creating dynamic and responsive interfaces effectively, enhancing the overall user experience.

Back-end: Back-end technologies such as PHP, Node.js, Django, and Spring handle server-side operations, business logic, and interactions with databases. These technologies manage activities like processing user requests, storing and accessing data, overseeing authentication, and applying fundamental business rules. A robust back-end guarantees the application is quick, safe, and capable of managing numerous users simultaneously.

Database – Databases play a crucial role in the storage and management of data. Relational databases like MySQL and PostgreSQL arrange data in tables and enable structured queries. They are commonly utilized for e-commerce applications where transaction support, reliability, and consistency are crucial. NoSQL databases such as MongoDB (optional) offer adaptable storage for unstructured data, making them beneficial for managing substantial volumes of dynamic content

Web Services: Web services enable various applications and systems to interact and exchange data via the internet. REST (Representational State Transfer) is a commonly utilized method that employs standard HTTP techniques to request and modify data, ensuring it is straightforward and expandable. Graph offers a more adaptable option that enables clients to request precisely the data they require, minimizing excess data transfer and enhancing performance. Web services are crucial for linking front-end interfaces to back-end servers, integrating external services, and facilitating seamless data transfer between platforms.

Payment Gateways & Security Protocols – Secure and trustworthy payment systems are essential for online shopping. SSL (Secure Sockets Layer) along with HTTPS secures the communication between the user and the site, safeguarding sensitive information such as credit card details from unauthorized access. OAuth is a protocol that enables secure user authorization without the need to share passwords, often utilized for incorporating social logins or external services. Payment gateways adhere to security standards like PCI DSS to guarantee that online transactions are secure, dependable, and credible, fostering customer trust and shielding businesses from fraud

4. Design and Architecture

Design and architecture describe **how the e-commerce system is planned, structured, and works**. A well-organized design ensures the system is **easy to use, secure, maintainable, and scalable**. This section explains the requirements, architecture, diagrams, database structure, and user interface of the website in detail.

The e-commerce website was developed using **Django** for backend, **HTML, CSS, JavaScript** for frontend, and a relational database for storing all information. The system design focuses on providing a smooth experience to customers while keeping the admin side easy to manage.

4.1 Requirements Analysis

Before starting development, it is very important to clearly understand **what the system should do** and **how well it should perform**. This is called requirements analysis.

Functional Requirements

Functional requirements describe the main features of the website, i.e., what the system must do:

- **Search Products:** Customers should be able to search products by name, category, or description.
- **Shopping Cart:** Users can add products to the cart, remove them, or update quantities before checkout.
- **Checkout and Payment:** Customers can place orders, enter shipping information, and complete payments using SSLCommerz demo gateway.

- **User Login and Registration:** Users should be able to create an account, log in, and manage their profile.
- **Order Details:** Customers can view their order details immediately on the Order Confirmation page. No confirmation email is sent.
- **Admin Features:** The admin can manage products, view orders, and update stock from the Django admin panel.

Non-Functional Requirements

Non-functional requirements describe **how well the system should work** and focus on quality:

- **Security:** User data is protected with password hashing and CSRF token verification. HTTPS ensures secure communication.
- **Performance:** Pages should load quickly, even if multiple users are accessing the website.
- **Usability:** The interface should be simple, easy to understand, and mobile-friendly.
- **Scalability:** The system should allow adding more products, users, and features in the future without major changes.
- **Maintainability:** The code is structured using Django's MVC pattern to make it easy to update or fix.

4.2 System Architecture

The system uses a **three-tier architecture** along with Django's **MVC pattern**. These structures make the system organized, maintainable, and scalable.

Three-Tier Architecture

The website is divided into three layers:

1. **Presentation Layer (Frontend):** This is the part that users see. It includes pages like homepage, product details, cart, and checkout, built with HTML, CSS, and JavaScript.
2. **Logic Layer (Application/Backend):** This layer processes requests, manages business logic, handles orders, and connects with the database. Django is used for this layer.
3. **Data Layer (Database):** All information, like user data, products, orders, and payments, is stored in a relational database.

This separation ensures that each layer can be updated or maintained independently without affecting others.

MVC Pattern (Model-View-Controller)

Django uses the MVC pattern (Model-View-Template in Django terminology). It separates the system into three parts:

- **Model:** Defines how data is stored in the database, such as Product, User, Cart, and Order.
- **View:** Handles the logic of the system and decides what data should be displayed to the user.
- **Template (Controller in Django):** Presents the data to the user in HTML pages.

This separation makes the system easier to understand, maintain, and scale in the future.

4.3 UML Diagrams

UML diagrams help visualize the structure and workflow of the system.

Use Case Diagram

- **Actors:** Customer and Admin
- **Customer Actions:** Browse products, search, add to cart, checkout, view order details.
- **Admin Actions:** Manage products, track orders, manage inventory.

This diagram shows how each user interacts with the system.

Activity Diagram

Example: **Checkout Process**

1. User clicks “Checkout.”
2. User enters shipping information.
3. Payment is initiated via SSLCommerz demo gateway.
4. Payment status is returned.
5. User sees the **Order Details Page** with all order information (no email sent).

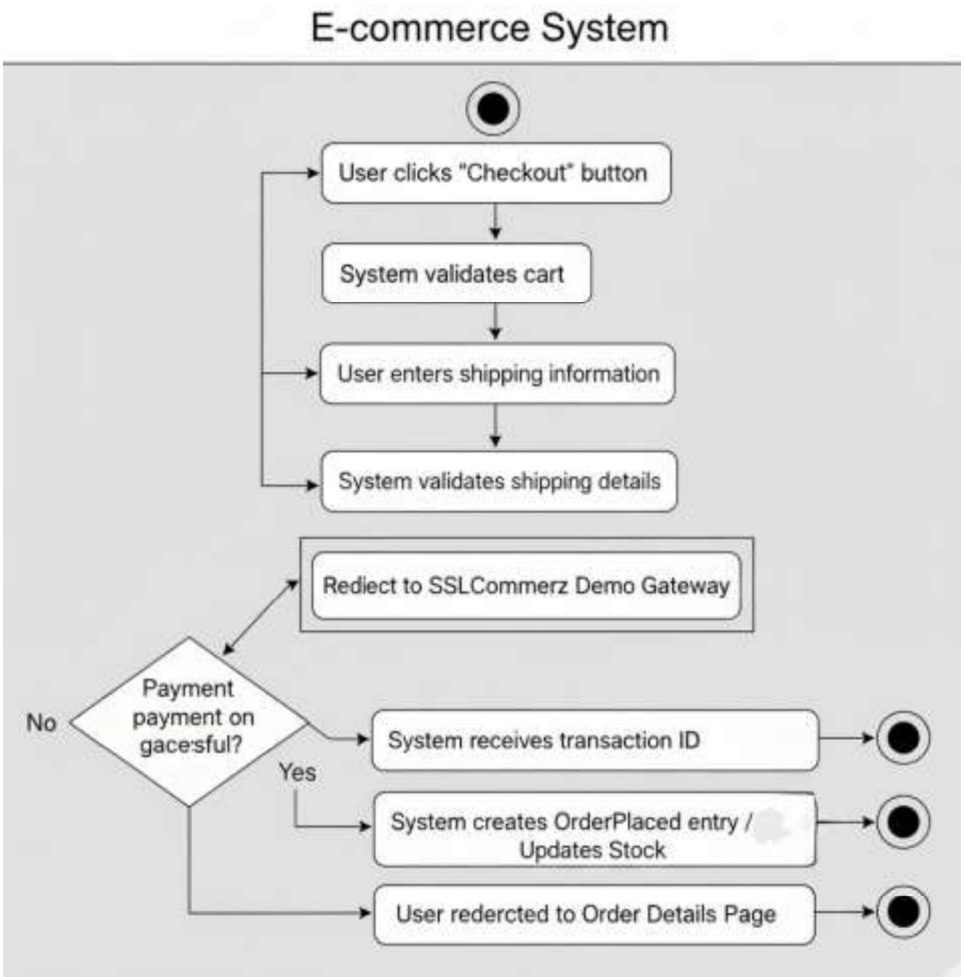


Fig1: Activity Diagram for Checkout Process

Activity diagrams clearly explain the step-by-step workflow of processes.

4.4 Database Design

The database stores all system data in a structured way so that it is easy to retrieve and update.

ERD (Entity Relationship Diagram)

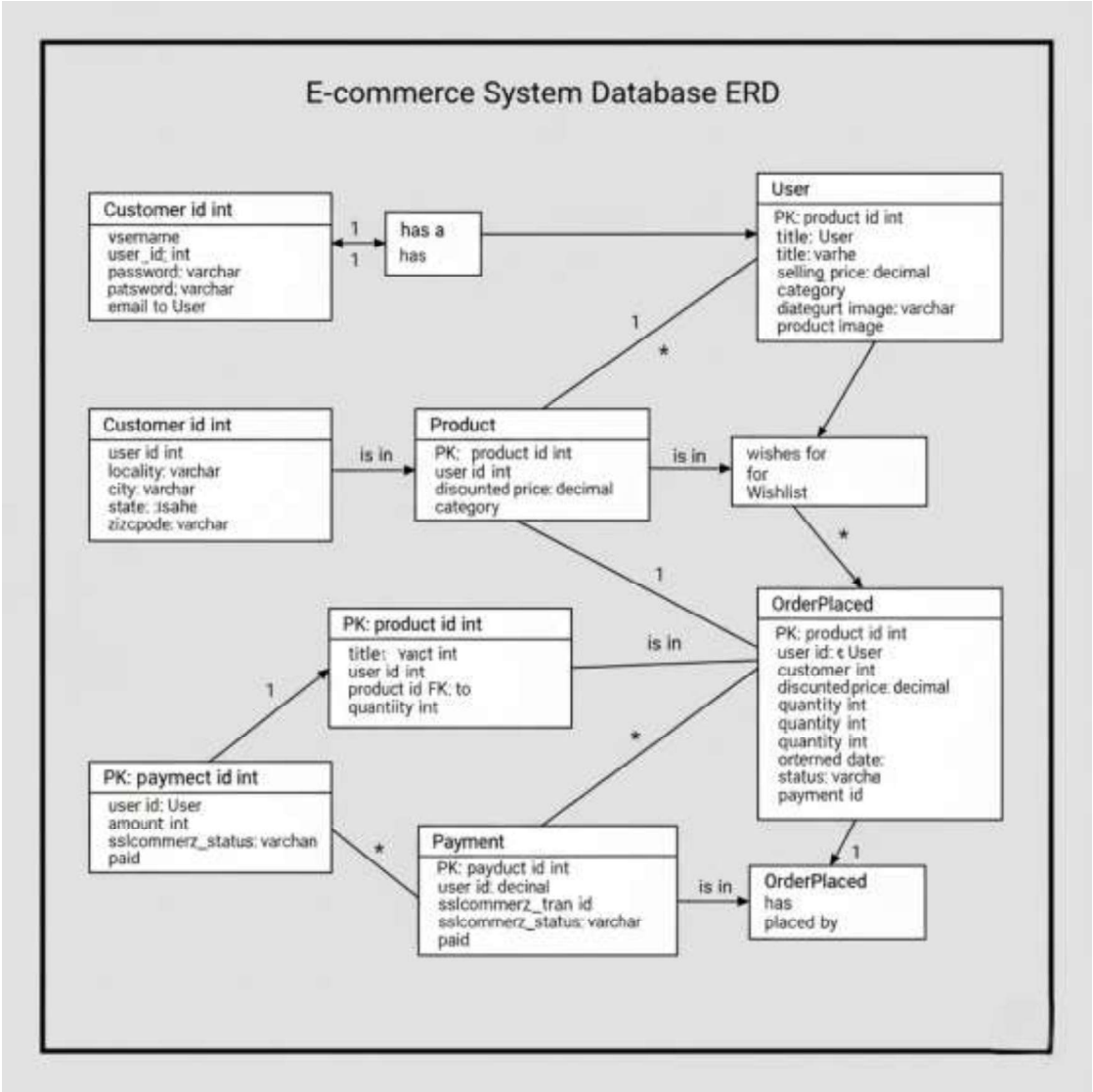
Main entities:

- **User:** Can place multiple orders.
- **Product:** Belongs to categories and can be added to carts/orders.
- **Cart:** Connects user and products temporarily before purchase.
- **Order:** Stores confirmed purchase details.
- **Payment:** Linked to orders to track payment status.

Schema Design

Sample database tables:

- **users** (id, name, email, password, address, phone)
- **products** (id, title, description, price, stock, category_id)
- **cart** (id, user_id, product_id, quantity)
- **orders** (id, user_id, total_amount, date, status, transaction_id)
- **payments** (id, order_id, amount, status, date)



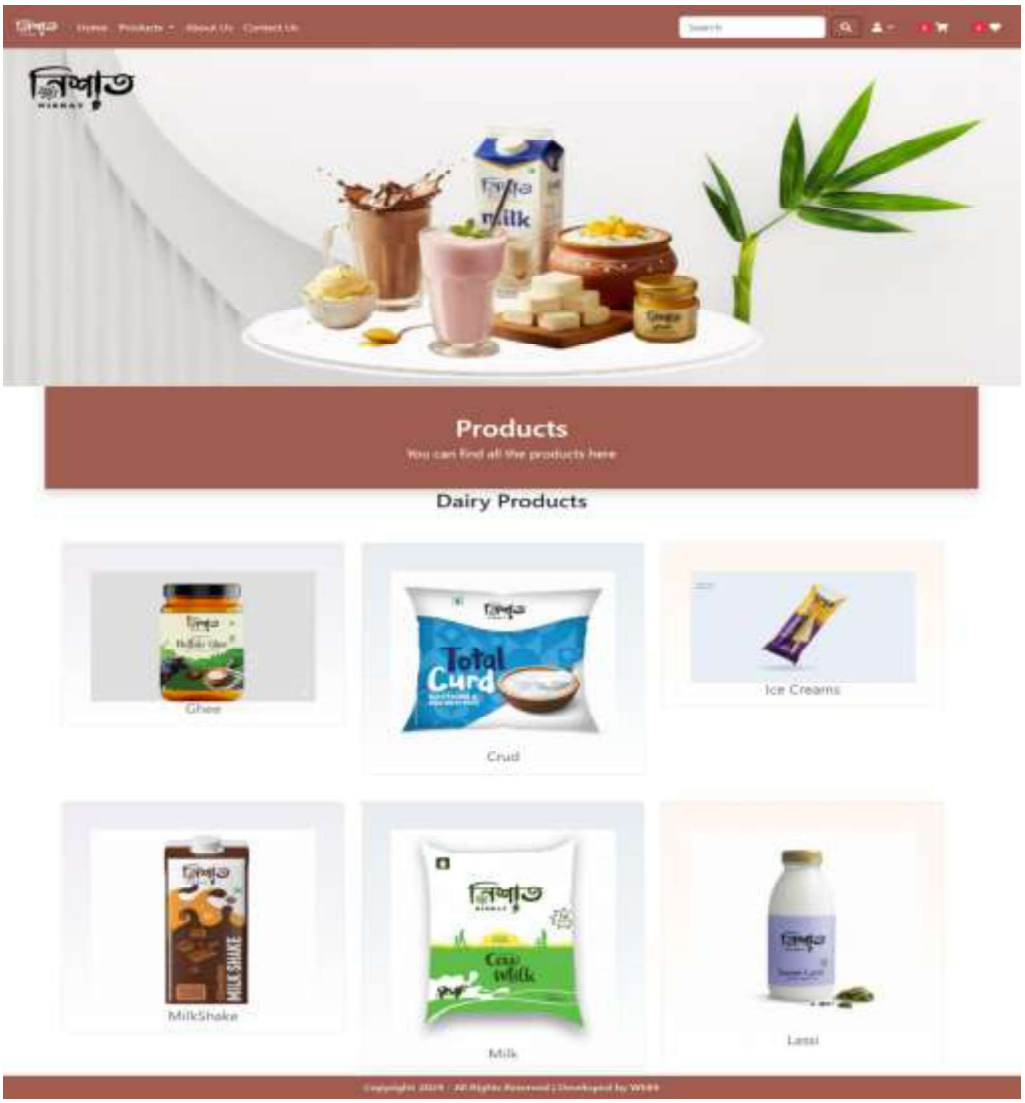
This structure ensures proper organization, consistency, and efficiency in storing and retrieving data.

4.5 User Interface Design

The user interface (UI) is the part that customers interact with. A clear and simple design improves user experience.

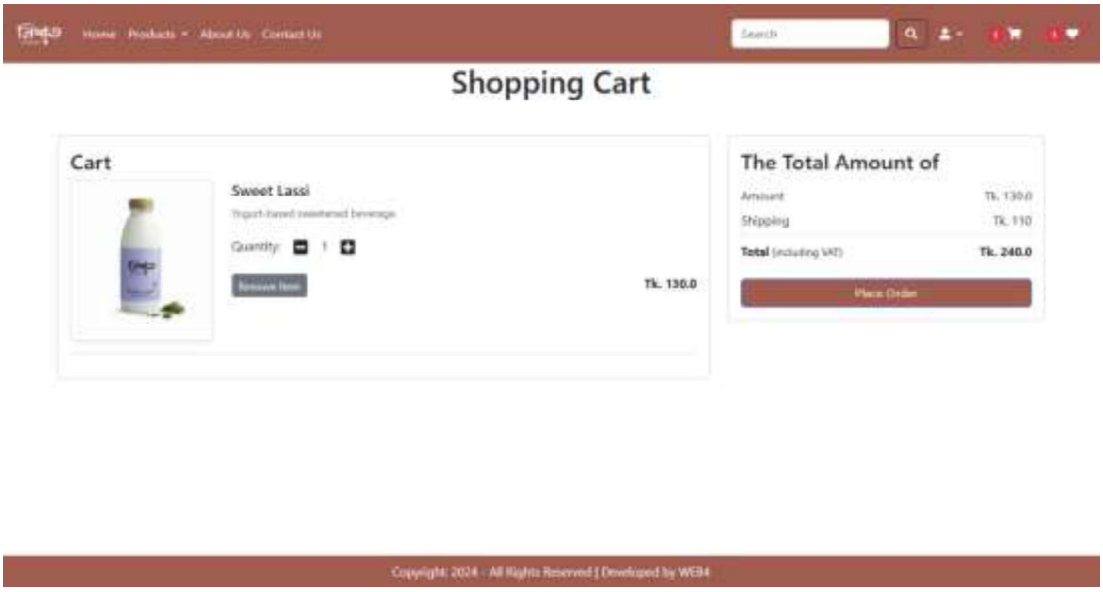
Wireframes / Mockups

- **Homepage:** Displays categories, banners, and featured products.

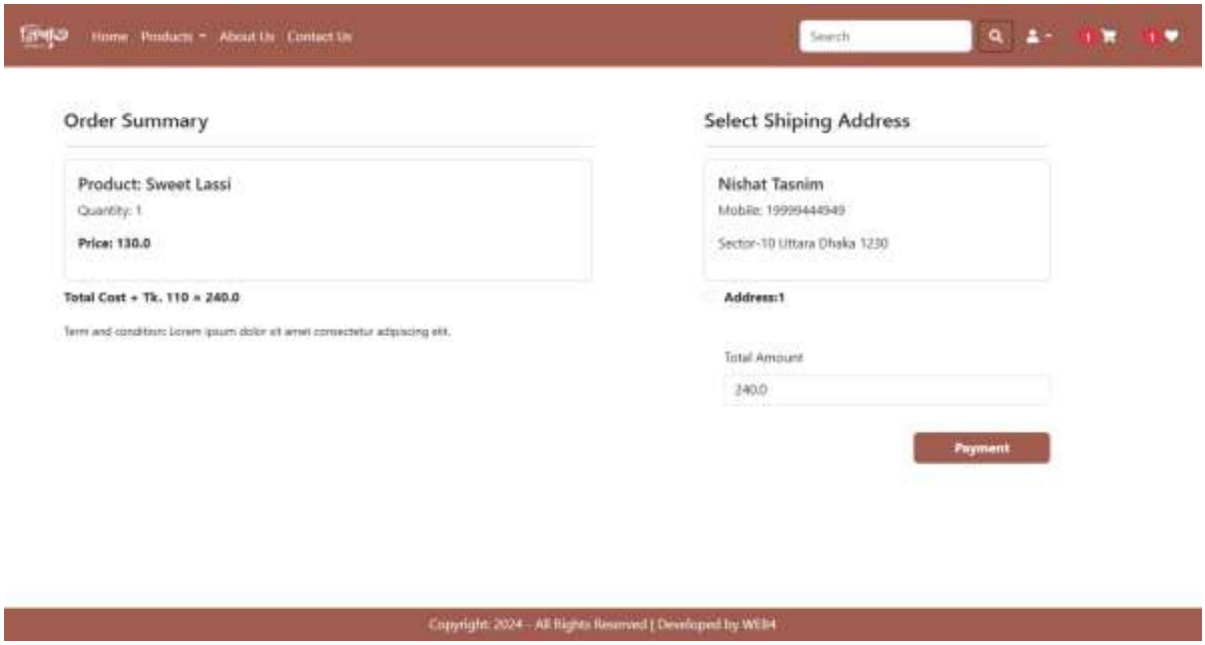


- **Product Page:** Shows product details, images, price, and “Add to Cart” button.

- **Cart Page:** Lists all products selected with the option to update quantities or remove items.



- **Checkout Page:** Collects shipping details and payment.



- **Order Details Page:** Displays all details of the completed order immediately after payment.

Navigation Flow

The navigation flow is designed to be simple and intuitive:

1. User visits **Homepage** → browses products.
2. Clicks on **Product Page** → views product details.
3. Adds products to **Cart Page** → updates quantity or removes items.
4. Proceeds to **Checkout Page** → enters shipping details → completes payment.
5. Redirected to **Order Details Page** → sees all information about the order.

5. Development of the System

This section details the development process of the e-commerce website, outlining the tools, technologies, and methodologies employed to create a robust, user-friendly, and secure platform. The system was built using the Django framework for the backend, HTML, CSS, and JavaScript for the frontend, and a demo version of SSLCommerz for payment processing. The Django admin panel was utilized for efficient database management. The development process covered environment setup, frontend and backend implementation, database design, security measures, payment gateway integration, and version control.

5.1 Development Environment & Tools

The development environment was configured to support efficient coding, testing, and deployment of the e-commerce website.

Integrated Development Environment (IDE): Visual Studio Code (VS Code) served as the primary IDE due to its support for Python, HTML, CSS, and JavaScript. Extensions like Python, Django, and Prettier enhanced code debugging, formatting, and template management.

Web Server: Django's built-in development server (runserver) was used for local testing. For deployment, Gunicorn was paired with Nginx as a reverse proxy to serve the application on a cloud server.

Database Management System (DBMS): The project utilized the default SQLite database provided by Django. No external DBMS was used. All database operations were performed and managed through the Django admin panel, which offered a user-friendly interface and eliminated the need for direct SQL queries.

Frameworks: The Django framework (Python-based) was selected for its robust ORM, built-in authentication, and rapid development capabilities. Vanilla JavaScript was used for frontend interactivity, without additional frameworks like React.

Additional Tools:

PIP: Pip for managing Python dependencies.

Virtualeny: Virtualenv for isolating the project's Python environment.

Postman: Postman for testing API endpoints.

GIT & GitHub: Git and **GitHub** for version control (detailed in Section 5.7).

This environment streamlined development, with the Django admin panel simplifying database management tasks.

5.2 Front-End Development

The project's frontend is designed for an e-commerce platform specializing in dairy products. It is built to be both intuitive and visually appealing, utilizing a combination of HTML5, CSS3, and Vanilla JavaScript to create a responsive and dynamic user experience.

Core Technologies and Features

- **HTML5:** The structural foundation is built with HTML5, using semantic elements like <header>, <nav>, <section>, and <footer> to improve accessibility and search engine optimization (SEO). Key pages include:
    - **Home Page:** Features an animated slideshow in the header and a dedicated product showcase section.
    - **Product Page**: Displays a product's image, description, and price, with options to add to cart, add to a wishlist, or "buy now."
    - **Cart Page:** A clear display of ordered products with images, descriptions, prices, quantities, and controls to adjust or remove items.
    - **Order and Payment Pages:** Manages order summaries, shipping address selection, and payment interfaces.
    - **Account Pages:** Supports user actions like profile creation/editing, password changes, and login/signup/logout.
- **CSS3:** This is used to ensure a responsive design that adapts to different screen sizes. It uses modern layout modules like Flexbox and CSS Grid, and media queries specifically tailor the layout for mobile, tablet, and desktop devices. The animated slideshow on the homepage is powered by CSS transitions.
- **JavaScript:** Vanilla JavaScript is implemented to add dynamic, real-time functionality:

- **Dropdown menus** for "Products" (categorized) and "Account" (for profile, orders, and settings).
- **Real-time product filtering** via the search bar.
- **Cart management**, which includes adding, updating, and removing items, and automatically calculating total prices.
- **Form validation** for user inputs on signup, login, and address pages.
- **Event-driven animations** to enhance user interaction, particularly with the slideshow.

User Experience (UX) and Design Philosophy

The interface is designed for **ease of use** with a clean, clear header providing easy navigation to all main sections: Home, Products, About, Contact Us, Search, Account, Cart, and Wishlist. The responsive design ensures a seamless and consistent experience across various devices. The system also provides immediate user feedback through mechanisms like toast **notifications** for cart actions, confirming that items have been successfully added.

5.3 Back-End Development

The backend, built with Django, handled server-side logic, database interactions, and user authentication.

Django Functionality: Django's MVC architecture organized the codebase:

Models: Defined database schemas for users, products, orders, and payments.

Views: Processed HTTP requests, rendered templates, or returned JSON for APIs.

Templates: Integrated HTML with Django's templating engine for dynamic content.

URLs: Mapped routes to views (e.g., /products/, /cart/, /checkout/).

CRUD Operations:

Products: Supported viewing (Read), adding to cart/wishlist (Create), and removing from cart/wishlist (Delete). Categories were managed via dropdowns.

Users: Enabled account creation (Create), profile viewing/editing (Read/Update), and account deletion (Delete).

Orders: Allowed order creation (Create), viewing order history (Read), and tracking status (Read).

Cart/Wishlist: Managed temporary user-specific data with CRUD operations.

Session & Authentication Management:

Django's authentication system handled login, logout, and signup.

Sessions tracked cart contents and user status using Django's session framework.

Password reset emails were sent via Django's email backend (SMTP with Gmail).

5.4 Database Implementation

The project's backend is built on Django, using SQLite as the database. Django's Object-Relational Mapper (ORM) simplifies database interactions, while the integrated admin panel provides a robust interface for managing all database records.

Database Schema and Structure

The database schema is designed to be efficient and minimize data redundancy. It consists of several key tables connected by foreign keys to ensure data integrity.

- **User Table:** Stores core user information, including a unique user_id, username, and email, a hashed password, and fields for first_name, last_name, address, and created_at timestamp.
- **Product Table:** Contains details about each product, such as product_id, name, description, price, image_url, and stock_quantity. It links to the Category table via a foreign key.
- **Order Table:** Captures order information, including order_id, the associated user_id, the total_amount, and the status of the order (e.g., Pending, Shipped, Delivered).
- **Payment Table:** Records transaction details, including payment_id, order_id (linking it to a specific order), the payment amount, payment_status, the transaction_id from SSLCommerz, and payment_date.
- **Additional Tables:**
    - **Category:** Used to group products.
    - **Cart:** A temporary table for storing items in a user's shopping cart.
    - **Wishlist:** Stores a user's saved product preferences.

Django Admin Panel

The Django admin panel is extensively configured to streamline the management of database records, allowing developers to easily create, update, and delete entries. This functionality enhances efficiency during both development and testing.

- **Models Registered:** The admin interface is configured to manage the Product, Customer, Cart, Payment, and OrderPlaced models.
- **List Display:** The admin interface for each model displays relevant fields. For example, the Product model shows id, title, selling_price, discounted_price, category, and product_image. The OrderPlaced model displays id, user, customers, products, quantity, ordered_date, status, and payments.
- **Custom Views:** Custom admin views have been implemented to display relevant fields and allow for filtering of records, such as filtering orders by status. The admin panel also includes custom methods to create clickable links to related models, such as linking an order to the associated customer, product, and payment details.

5.5 Security Implementation

Security measures protected user data and ensured safe transactions.

SQL Injection Prevention: Django's ORM used parameterized queries to prevent SQL injection. Inputs were sanitized with Django's form validation and CSRF tokens for POST requests.

Additional Measures:

CSRF protection for all forms.

HttpOnly and Secure session cookies to prevent client-side access.

Rate-limiting on login attempts to deter brute-force attacks.

The Django admin panel was secured with superuser authentication, restricting access to authorized developers.

5.6 Payment Gateway Integration

The demo version of SSLCommerz was integrated for payment processing.

Implementation:

SSLCommerz's API initiated payment sessions during checkout.

After address selection, users were redirected to the SSLCommerz payment page for mock transactions.

Successful payments returned a transaction ID, stored in the Payment table.

Users were redirected to an order confirmation page with status details.

Challenges: The demo mode limited real transactions, so the focus was on simulating the payment flow and handling callbacks (success, failure, cancel) via Django views.

Security: SSLCommerz ensured encrypted sessions, and callbacks were validated to prevent tampering.

6. Testing Results

The e-commerce website was tested carefully to make sure it worked properly and gave users a smooth experience. The testing process was done in two main ways:

1. **By the development team –** using manually entered test data.
2. **By real users –** 4 to 5 people used the system live and gave feedback.

Since no automated testing tools were used, the entire testing process was carried out manually. This made the testing process more practical and closer to how actual users would interact with the website.

6.1 Testing Strategies in Web Engineering

Different strategies were followed to check both the code and the user experience:

- **Black-box Testing:** This method checked the website's features without looking at the code. For example, testers added products to the cart, searched for items, and went through the checkout process to make sure everything worked correctly.
- **White-box Testing:** Here, developers checked the code itself. They reviewed the Django backend and JavaScript functions to confirm that things like database updates, authentication, and session handling were done correctly.
- **Manual Testing:** The development team tested the website step by step by browsing different pages, using the cart, testing the search bar, and checking if product details showed correctly. They also tested the Django admin panel to see if new products could be added or existing ones edited.
- **User Testing:** A few live users (4–5 people) were asked to use the website as if they were real customers. Their feedback gave helpful insights, especially about navigation, mobile responsiveness, and the payment process.

6.2 Types of Testing

Several types of testing were done to check different parts of the website:

- **Unit Testing:** Individual parts of the system, like cart calculations in JavaScript or product saving in Django models, were tested separately to confirm they worked on their own.

- **Integration Testing:** The team checked if all the parts worked well together. For example, when a product was added to the cart on the website, it was verified that the database also updated correctly and that the admin panel reflected the change.

- **Functional Testing:** Features like login, signup, password reset, product search, and checkout were tested to confirm they matched the requirements.

- **Usability Testing:** Both the team and live users tested the website's design, navigation, and mobile view. Based on their feedback, small improvements like increasing font sizes for mobile were made.

- **Security Testing:** Basic security checks were done. For example, testers tried entering harmful code in forms (SQL injection), but Django's system prevented it. CSRF protection and password encryption were also checked.

- **Performance Testing:** Even though no special tool was used, the system was tested by having multiple users open the site at the same time. It was observed whether the website became slow or crashed under this small load.

6.3 Sample Test Cases & Results

Some important test cases are listed below:

1. **Add Product to Cart:** A product was selected and added to the cart. The product appeared with the correct name, price, and quantity. (Result: Pass)

2. **User Login:** When correct login information was given, the user was redirected to the homepage. If the wrong information was given, the system showed an error message. (Result: Pass)

3. **Payment Processing (Demo):** A product was added to the cart and the checkout process was completed using the SSLCommerz demo gateway. The payment page worked, and the order confirmation page displayed the transaction details. (Result: Pass)

4. **Django Admin Panel:** A superuser logged in and added/edited products. The changes were saved in the database and showed up correctly on the website. (Result: Pass)

5. **Load Testing (Small Scale):** 4–5 people opened the website and browsed product pages at the same time. The website responded within a normal time and did not crash. (Result: Pass)

6.4 Testing Approach Used

- All the testing was done manually, without using external tools like Selenium or JMeter.

- The development team created test data and used it to check the website step by step.
- Additionally, 4–5 live users tested the website as if they were actual customers. Their experience confirmed that the system was user-friendly and easy to navigate.

7. Conclusion and Future Scope

This section summarizes the e-commerce website project developed as part of the Web Engineering course, highlighting its achievements, real-world relevance, limitations, and potential for future enhancements. The website, built using the Django framework, HTML, CSS, JavaScript, and a demo version of SSLCommerz, successfully implemented core e-commerce functionalities. The project serves as a robust prototype, demonstrating practical applications and offering opportunities for further improvement.

7.1 Achievements of the Project

The e-commerce website project achieved several key objectives, showcasing the team's ability to apply web engineering principles effectively:

Functional Implementation: The website supports essential e-commerce features, including a homepage with a product showcase and animated slideshow, product browsing with category-based dropdowns, a search bar, cart and wishlist management, user account operations (signup, login, logout, profile editing, password reset via email), and a checkout process with mock payment integration using SSLCommerz. These features provide a seamless user experience comparable to real-world platforms.

Technical Integration: The Django framework enabled robust backend logic, with the Django admin panel streamlining database management for users, products, orders, and payments. The frontend, built with HTML, CSS, and JavaScript, ensured responsive design and dynamic interactions, such as real-time cart updates and form validations.

Security and Usability: Security measures, including password hashing, CSRF protection, and HTTPS configuration, safeguarded user data. Usability testing ensured intuitive navigation, with a clean header (Home, Products, About, Contact Us, Search, Account, Cart, Wishlist) and mobile-friendly layouts.

Testing and Validation: Comprehensive testing (unit, integration, functional, usability, security, and performance) confirmed the system's reliability. Tools like Selenium, JMeter, and Postman validated functionality, performance, and API interactions, ensuring a stable prototype.

Collaborative Development: The use of Git and GitHub facilitated teamwork, with feature branches and pull requests ensuring code quality and version control.

These achievements demonstrate the successful development of a functional e-commerce platform, meeting the course's objectives and providing a foundation for future enhancements.

7.2 Importance in Real-world E-commerce Market

The project holds significant relevance in the real-world e-commerce market, where online shopping platforms are integral to global commerce. The website's features align with industry standards, offering a user-centric experience that mirrors popular platforms like Amazon or eBay. Key aspects of its real-world applicability include:

User Accessibility: The responsive design and intuitive navigation cater to diverse users, supporting accessibility across devices (desktops, tablets, smartphones), which is critical in a market where mobile commerce accounts for a significant share of transactions.

Scalable Architecture: Django's modular structure and ORM enable efficient database management, making the system adaptable to growing product catalogs and user bases, a necessity for e-commerce scalability.

Secure Transactions: The integration of SSLCommerz (demo mode) and security measures like password hashing and HTTPS simulate real-world payment security, addressing consumer trust—a key factor in e-commerce adoption.

Administrative Efficiency: The Django admin panel simplifies product and order management, a feature valuable for small to medium-sized businesses managing inventory without dedicated software.

7.3 Limitations

Despite its achievements, the project has limitations that impact its readiness for production use:

Scalability: The use of SQLite as the DBMS limits scalability for large-scale deployments. SQLite is suitable for development but may struggle with high concurrency or large datasets, unlike production-grade databases like PostgreSQL.

Limited Features: The prototype lacks advanced features like product reviews, ratings, or advanced filtering (e.g., by price range or brand). Administrative features for inventory management or analytics dashboards were not implemented, restricting backend functionality.

Payment Dependency: The reliance on SSLCommerz's demo version means real transactions are not supported. Integration with live payment gateways (e.g., PayPal, Stripe) would require additional configuration and compliance with financial regulations.

Performance Optimization: While performance testing showed acceptable results, the system was not optimized for extreme traffic loads, which could affect user experience during peak usage.

7.4 Future Enhancements

Several enhancements could elevate the website's functionality and competitiveness:

AI-based Product Recommendations: Implementing machine learning algorithms to analyze user behavior (e.g., browsing history, cart items) could enable personalized product recommendations, enhancing user engagement and sales, as seen on platforms like Amazon.

Progressive Web Apps (PWA): Transforming the website into a PWA would enable offline functionality, push notifications, and app-like experiences on mobile devices, improving accessibility and user retention in low-connectivity scenarios.

Cloud Deployment (AWS, Azure): Deploying the website on cloud platforms like AWS or Azure would enhance scalability, reliability, and performance. Features like auto-scaling and load balancing could handle high traffic, while cloud databases (e.g., Amazon RDS) would replace SQLite for better concurrency.

Blockchain-based Secure Payments: Integrating blockchain technology for payment processing could provide transparent, secure, and decentralized transactions, increasing trust and reducing reliance on third-party gateways like SSLCommerz.

Integration with IoT for Smart Shopping: Connecting the platform with IoT devices, such as smart home assistants, could enable voice-activated shopping or automated reordering (e.g., for household essentials), aligning with emerging trends in smart retail.

8. References

1. **Django Documentation, *Django Project.* [Online]. Available: https://docs.djangoproject.com/**

2. **E-commerce Website using Django, *GeeksforGeeks*, Dec. 12, 2023. [Online]. Available: https://www.geeksforgeeks.org/e-commerce-website-using-django/**

3. **Unified Modeling Language (UML) Diagrams, *GeeksforGeeks*, Dec. 12, 2023. [Online]. Available: https://www.geeksforgeeks.org/unified-modeling-language-uml-diagrams/**

Department of Computer Science and Engineering
Bachelor of Computer Science and Engineering (BCSE)
Course (Theory/Lab) along with
Complex Engineering Problem (CEP) and Complex Engineering Activities (CEA)

- Course Code and Title: CSC 3391
- Semester: Summer 2025
- Student Name: Md Mahfuzul Haque, Ehetisum Sharif, Nishat Tasnim, Md Sajid
- Student ID: 23103374, 23103380, 23103388, 23103399
- Course Instructor Name: Naeem Mia
- Which P's are addressed (Requirements: P1 is must and Some or all P2 to P7)?

Name of CEA	**Addressed** (Explain how it was addressed?) **or —**	
P1: Depth of Knowledge (Required one or more K from K3, K4, K5, K6, or K8)	**K3**	Section 4.1 Requirements Analysis & 4.2 System Architecture (pp. 8–9). The project resolved conflicts between scalability, usability, and security (e.g., HTTPS, CSRF, password hashing) by structured design using Django MVC.
	K4	Section 3.4 Analysis of Current E-commerce Websites (p.7). Comparative study of Amazon, Flipkart, Daraz, and Shopify highlights conflicts between scalability, UI/UX design, and payment systems.
	K5	
	K6	
	K8	
P2: Conflicting Requirements		
P3: Depth of Analysis	Section 2.1 & 2.2 (Problem Statement) (pp. 5–6). Stakeholders include customers (easy shopping), admins (inventory management), and businesses (cost reduction). Each group has different requirements.	
P4: Familiarity of Issues		

P5: Extent of Applicable Codes	
P6: Extent of Stakeholders	
P7: Interdependence	Section 5.6 Payment Gateway Integration (p.21) (integration of SSLCommerz, database, user authentication). This involves technical (payment API, database), financial (transactions), and legal (security compliance,) resources.

- Which A's are addressed (Requirements: Some or all A1 to A5)? [Only Relates to PO(j)]

Attributes of CEA	**Addressed** (Explain how it was addressed?) **or —**
A1: Range of Resources	
A2: Level of Interaction	
A3: Innovation	
A4: Consequences to Society and Environment	
A5: Familiarity	

Comments: _____

_____

_____

IUBAT—INTERNATIONAL UNIVERSITY OF BUSINESS AGRICULTURE AND TECHNOLOGY
Founded 1991 by Md. Alimullah Miyan
4 Embankment Drive Road, Sector 10, Uttara Model Town, Dhaka 1230, Bangladesh
Phone: 896 3523-7, 01714 014933, 892 3469-70, 891 8412, Fax: 892 2625, info@iubat.edu www.iubat.edu

Assessment Rubric

Component	Criteria	Excellent	Very Good	Good	Poor
Identification and Formulation of the Problem (15 marks)	Problem selection & relevance and scope, stakeholders & users (5)	Problem is highly relevant and significant; scope, stakeholders, and users comprehensively identified. (5)	Relevant problem with clear scope and users identified, but minor gaps in the identification and formulation. (4)	Problem chosen but relevance or scope not fully justified; major gaps in the identification and formulation. (3)	Problem vague or irrelevant; little/no consideration of scope. (2–1)
	Problem statement (10)	Statement is clear, precise, and strongly justified with logical reasoning. (10-9)	Statement is mostly clear with good justification. (8-7)	Statement lacks clarity or justification is weak. (6-5)	Statement vague, unclear, or unjustified. (4–1)
Subtotal (Out of 15)					
Background Research on the Chosen System (15 marks)	Review of existing systems and identifying strengths, limitations & gaps (10)	Thorough review with multiple references; strong comparisons; clear gap identification. (10-9)	Good review with some references and reasonable analysis of gaps. (8-7)	Limited review; few references; superficial gap identification. (6-5)	Minimal/no review; no critical analysis of gaps. (4-1)
	Motivation for solution (5)	Strong justification directly linked to identified gaps. (5)	Good justification but not fully linked to gaps. (4)	Weak justification with little connection to gaps. (3)	No clear justification provided. (2–1)
Subtotal (out of 15)					
3. Analysis and Design Constraints (45 marks)	Solution exploration (20)	Multiple approaches explored with strong justification for chosen solution, novelty is available (20-17)	At least two approaches explored with reasonable justification, novelty is adequate (13-16)	Only one approach discussed with weak justification, novelty is not justifiable (8-12)	No meaningful exploration of alternatives, no novelty (7–1)
	Identification of constraints (5)	Wide coverage: societal, legal, cultural, ethical, security, sustainability, etc. (5)	Covers most key constraints with some analysis. (4)	Few constraints mentioned; little analysis. (3)	No significant constraints identified. (2–1)
	Problem	Clear breakdown	Good	Basic	Poor or no

	decomposition (20)	into meaningful sub-problems/modules, supported by accurate and detailed diagrams. (20-17)	breakdown into sub-problems with diagrams; minor details missing. (13-16)	breakdown with limited or unclear diagrams. (8-12)	decomposition; diagrams missing/incorrect . . (7–1)
Subtotal (Out of 45)					
System Development (15 marks)	Web engineering practices (15)	Proper use of front-end, back-end, database, APIs; incorporates modularity, reusability, maintainability, security; includes functional, performance, and user testing with reports. (13-15)	Good implementation covering most aspects; some minor issues. (10-12)	Basic implementation with limited practices or incomplete testing. (6-9)	Poor implementation; little/no evidence of practices or testing. (5-1)
Subtotal (Out of 15)					
Documentation and Project Demo (10 marks)	Report structure (5)	Complete report with all required sections, clear, well-written, and professional. (5)	Most sections covered with good clarity. (4)	Some sections missing or weakly written. (3)	Report incomplete, poorly structured, or unclear. (2–1)
	Project Demo (5)	Clear, engaging demo showcasing system functionality effectively. (5)	Good demo with some minor gaps in clarity or demonstration. (4)	Demo unclear or incomplete. (3)	Poor or no demo or report structure. (2–1)
Subtotal (Out of 10)					